

Data & Interface Specification Manual (v2.0)

November 09, 2025

This manual defines the exact formats, files, and contracts every team member must follow. **No deviations allowed** unless the group formally agrees on an update. Following this ensures all components are plug-and-play compatible.

1. IDs & Keys

- Every row in vectors, metadata, and queries is keyed by a unique `id:int64`.
- IDs must be contiguous from $0 \dots N-1$.
- Joins between files are only on `id`.
- **Validation:** no duplicates, no gaps.

2. Vector Store

- **File name:** `vectors.npy`
- **Shape:** (N, D)
- **dtype:** `float32`
- **Normalization:** L2-normalized ($\|x\|_2 \approx 1$).
- **Metadata file:** `vectors.meta.json` (JSON keys: N, D, normalized, model, created_utc, seed).

Rules

- The dataset and its metadata jointly define the canonical vector store.
- `vectors.npy` and `vectors.meta.json` **must** remain synchronized on all declared attributes (N, D, normalized).
- Query vectors must have the same dimension D and unit norm within tolerance $1e-3$.
- All reading and validation of vectors and metadata **must** follow §11 using the official loaders in `dataio.loaders`.
- Direct file access, ad-hoc normalization, or implicit schema inference is **not permitted**.

3. Metadata Table

- **File name:** `metadata.parquet`
- **Required columns**
 - `id:int64`
 - `state:string`
 - `city:string`
 - `stars:float32`
 - `review_count:int32`
 - `RestaurantsPriceRange2:int8`

- **Optional columns:** `categories`, `latitude`, `longitude`, `is_open`, `attributes.json`.
- **Rules**
 - `id`, `state`, `stars`, and `review_count` must be non-null.
 - IDs must be contiguous $0..N-1$ and sorted ascending.
 - Metadata must describe exactly the same N entities as `vectors.npy`.
 - All validation and type enforcement **must** follow §11 using `dataio.loaders.load_metadata(...)` and `dataio.validators.validate_metadata_table(...)`.
 - No local schema enforcement or implicit casting is allowed.

4. Queries

- **File name:** `queries.parquet`
- **Schema**
 - `qid:int32`
 - `qtext:string`
 - one of:
 - * `filters:json` (v1 style), or
 - * `filters_json:string` (v2 style; JSON object encoded as string)
 - `K:int32`
 - `label:string` ("strict" | "medium" | "broad")

Filter JSON example

```
{
  "state": {"eq": "FL"},
  "stars": {"ge": 4.0},
  "review_count": {"between": [50, 200]},
  "RestaurantsPriceRange2": {"in": [2, 3]},
  "categories": {"like": "Pizza"}
}
```

Supported operators: `eq`, `ne`, `in`, `between`, `le`, `gt`, `lt`, `like`, `lat_between`, `lon_between`.

Rules

- Queries must reference only valid metadata fields and supported operators (§12).
- All parsing, schema checking, and allowed-ID generation **must** follow §11 using the canonical functions in `dataio.validators`.
- K must satisfy $1 \leq K \leq N$ as enforced by `validate_K(...)`.
- Ad-hoc JSON parsing, operator definitions, or filter logic outside §11 are strictly prohibited.

5. Query Vectors

This project currently maintains **two artifact buckets** under `artifacts/`:

- `artifacts/v1/` — legacy / archived test set created during early development.
- `artifacts/v2/` — current, selectivity-aware test set used for main experiments.

`v1` is preserved for reproducibility but is **not** the main testing set going forward.

5.1 v1: on-the-fly embedding (archived profile)

Artifacts in `artifacts/v1/` were created from earlier, human-readable queries (e.g. “good pizza”, “authentic sushi”). For this bucket it is acceptable for the harness to embed queries at runtime.

Rules:

- The harness embeds `qtext` at runtime using the **same model** recorded in `artifacts/v1/vectors.meta.json`.
- Embedded query vectors **must** be L2-normalized `float32`.
- The harness **must** validate that the query dimension D matches the dataset vector dimension D from `vectors.npy`.
- `artifacts/v1/` is to be treated as **immutable** once committed (archive).

Example config intent:

```
query_embedding:
  mode: on_the_fly
  model_from_vectors_meta: true
  artifacts_bucket: artifacts/v1
```

5.2 v2: pre-embedded, filter-driven queries (required)

Artifacts in `artifacts/v2/` are generated from metadata to hit specific **selectivity buckets** (via `artifacts/make_queries.py`). In this bucket, the meaningful part of the query lives in `filters_json`, while `qtext` is often just a label like `auto-strict`. Because of that, **runtime embedding of `qtext` is not meaningful and must not be used**.

For `artifacts/v2/`, the harness **must** load pre-materialized query vectors.

Files in this bucket:

- `metadata.parquet`, `vectors.meta.json`, and `vectors.npy` — inherited from v1 build logic.
- `queries.parquet` — **generated** by `artifacts/make_queries.py`; contains `qid`, `qtext` (label), and `filters_json` (authoritative filter).
- `query_vectors.npy` — **generated** by `artifacts/make_query_vectors.py`; shape (Q, D), `float32`, L2-normalized; indexed by `qid`.

Standard harness usage (`src/harness/run.py`):

1. The user runs with `--artifacts v2`.
2. The harness loads **all** artifacts from `artifacts/v2/` (vectors, metadata, queries).
3. The harness detects that `query_vectors.npy` exists in the same bucket.
4. The harness uses `query_vectors[qid]` directly and **does not** embed `qtext` at runtime.
5. Filters are parsed from the `filters_json` column and applied as usual.

Rules:

- v2 runs **must** supply `query_vectors.npy` in the same folder as `queries.parquet`.
- The harness **must not** fall back to embedding `auto-*` strings for v2.
- `query_vectors.npy` must be L2-normalized and have the same D as `vectors.npy`.
- `qid` must be contiguous and must match the row order in `query_vectors.npy`.

Regeneration policy:

- To create or refresh v2 queries: run `artifacts/make_queries.py` (writes `artifacts/v2/queries.parquet`).
- To create matching vectors: run `artifacts/make_query_vectors.py` (writes `artifacts/v2/query_vectors.npy`).
- These scripts may be modified locally to adjust buckets/tolerance, **but** regenerated v2 artifacts (and script changes) must **not** be pushed to the main branch unless approved by the whole group, to preserve experiment comparability.

5.3 No mixed modes

- A single run must use **one** artifact bucket and **one** query-vector strategy.
- Do **not** run v2 queries against v1's on-the-fly path.
- Do **not** partially pre-embed some queries and embed the rest at runtime.

6. Backend Interface

Every backend (prefilter, postfilter, hybrid, exact) must implement:

```
class SearchBackend:
    name: str
    def search(self, qvec: np.ndarray, filters: dict, K: int) -> tuple[list[int], dict]:
        """
        Returns:
            ids: list[int] # Top-K ids in descending IP order
            stats: dict # see Stats Contract
        """
```

Hybrid backends (`/backends/hybrid_backend.py`) are considered extensions of this interface. They must subclass `SearchBackend`, define `name = "hybrid"`, and implement `search(qvec, filters, K)` in an interchangeable manner. All return values and stats follow the same contract as other backends.

7. Hybrid Backend Contract

Hybrid backends combine approximate nearest-neighbor (ANN) retrieval with metadata filtering. They extend the `SearchBackend` interface (§6) and define the standard for FAISS-based hybrid search.

7.1 Scope

Applies to all modules under `/baselines/hybrid/` and `/backends/hybrid_backend.py`. Hybrid systems integrate FAISS with allow-list filtering (`faiss.IDSelectorBatch`) and must comply with this section in addition to §§6, 8–11.

7.2 Interface Compliance

Hybrid backends **must** subclass `SearchBackend` (§6) and define `name = "hybrid"`. They implement the same `search(qvec, filters, K)` interface and return `(ids, stats)` in identical structure, allowing drop-in interchangeability with other backends.

7.3 Filter Enforcement

Filters **must** be parsed and validated according to §11 before use. Hybrid backends **must** restrict FAISS search to the validated allow-list (`allowed_ids`) using `faiss.IDSelectorBatch` or equivalent. Returned IDs **must** be a subset of the `allowed_ids`. Running FAISS without selector enforcement is **prohibited**.

7.4 Index Requirements

Hybrid backends **must** use FAISS indices built over the same normalized vectors defined in §2 and validated in §11. The index type **must** use inner-product metrics (`IndexFlatIP`, `IndexIVFFlat(IP)`), consistent with cosine similarity. For IVF indices, `nlist` and `nprobe` values **must** be logged (§8). Progressive-probe schedulers are allowed but each phase **must** log its effective `nprobe`.

7.5 Performance Path Rules

Hybrid backends **may** implement iterative or multi-phase probing, reranking, or staged evaluation. All intermediate operations **must** act only on candidates within the enforced allow-list, and the final Top-K must be selected strictly by inner-product score.

7.6 Stats & Logging

Hybrid backends **must** populate all standard stats fields defined in §8 and log results per §9. Any hybrid-specific details (e.g., `rerank_size`, `schedule_phase`, `retries`) **must** appear only under `extras: {}`. Results may be written to `/results/<run_id>/results_hybrid.jsonl`.

7.7 Artifact Compliance

Hybrid backends **must not** introduce new artifact file types or alter existing bucket structures. All hybrid runs operate over validated artifacts and immutability rules defined in §10 and §11.3.

8. Stats Contract

Every backend must return a `stats` dictionary with these keys:

- `latency_ms`: float
- `scored_vectors`: int
- `lists_probed`: int
- `nprobe`: int | null
- `kth_at_stop`: float | null
- `bound_at_stop`: float | null
- `notes`: str | null

If a field doesn't apply → set to null.

Hybrid backends must also populate all fields listed above. Any additional hybrid-specific runtime statistics (e.g., `schedule_phase`, `rerank_size`, `retries`) must appear only as a nested dictionary under `extras: {}` in the JSONL log.

- For Pre-filter: `scored_vectors` = `|allowed_ids|`
- For Post-filter: `scored_vectors` = `K` (oversampled candidate count)

9. Oracle Contract

- **Function**

```
def brute_force(qvec: np.ndarray, allowed_ids: np.ndarray, K: int) -> list[int]
```

- Must rank **only within allowed_ids** by inner product.
- Returns exact Top-K → gold standard for recall.

10. Logging Contract

- **Format:** JSONL only (one JSON object per line). CSV not permitted.
- **Approved path patterns:**
 - /results/<run_id>/results.jsonl
 - /results/<run_id>/pre_results.jsonl
 - /results/<run_id>/post_results.jsonl
- **Fields (fixed order, all required)**

field	type	example
qid	int32	7
method	string	“hybrid”
K	int32	10
latency_ms	float	9.74
recall_at_k	float	0.9
filter_selectivity	float	0.04
scored_vectors	int32	146
lists_probed	int32	8
nprobe	int32	16
kth_at_stop	float	0.7111
bound_at_stop	float	0.7094
notes	string	“skip empty lists”
timestamp_utc	string	ISO-8601
run_id	string	“wk1_dev_v1”

Hybrid backends may output to /results/<run_id>/results_hybrid.jsonl in addition to standard result files. This path is fully compliant and intended for direct comparison with **results_pre.jsonl** and **results_post.jsonl**. All field ordering and schema must remain identical to the standard format.

Rules

- Missing values → null.
- No extra top-level fields. For ablations, add nested **extras: {}**.
- Append-only. Runs may add rows but never overwrite.
- The harness must write fields **in this exact order**.
- **pre_results.jsonl** and **post_results.jsonl** are fully supported formats intended for separated backend evaluation and direct comparison. They follow the **exact same schema** and validation rules as **results.jsonl** and are considered **first-class log outputs** under this specification.
- If combined reporting is needed, these files may later be merged into **results.jsonl**, but separate storage is equally compliant.

11. File Layout

```
/artifacts/  
  artifacts.py  
  make_queries.py  
  make_query_vectors.py  
  /v1/  
    vectors.npy  
    vectors.meta.json  
    metadata.parquet  
    queries.parquet  
  /v2/  
    vectors.npy  
    vectors.meta.json  
    metadata.parquet  
    queries.parquet  
    query_vectors.npy  
  
/config/  
  run.yaml          # Currently empty, reserved for future use  
  
/results/  
  /week1/  
    results_dev.jsonl  
    results_full.jsonl  
  /week2/  
    exact_results.json  
    pre_results.jsonl  
    post_results.jsonl  
    latency_selectivity.png  
    recall_selectivity.png  
  
/src/  
  /backends/  
    exact_backend.py  
    pre_filter_backend.py  
    post_filter_backend.py  
    hybrid_backend.py  
    backend_interface.py  
  /baselines/  
    pre_filter.py  
    post_filter.py  
  /hybrid/  
    index.py  
    scheduler.py  
    search.py  
    selector.py  
  /harness/  
    run.py  
  /dataio/  
    loaders.py  
    validators.py  
  /eval/  
    oracle.py
```

```
metrics.py
selectivity.py
/plots/
  latency_selectivity.py
  recall_selectivity.py
/utils/
  logger.py
  timing.py
```

Rules:

- Artifacts are **immutable once committed**. If regeneration is needed, bump version (v2). Never overwrite v1.
- During development, intermediate baseline runs may output files such as `/results/weekX/results_pre.jsonl` or `/results/weekX/results_post.jsonl`, which must later be merged into the canonical `/results/<run_id>/results.jsonl`.

12. Validation Rules

All validations must be performed using the centralized modules in `src/dataio/loaders.py` and `src/dataio/validators.py`. No script, backend, or harness may implement independent validation logic.

12.1 Loader-Level Validation

- **Vectors and Metadata**
 - Artifacts **must** be read through `dataio.loaders.load_vectors(...)`, `loaders.load_vectors_meta(...)`, and `loaders.load_metadata(...)`.
 - These loaders **must** be the only entry points for reading `.numpy`, `.json`, or `.parquet` artifact files.
 - Any code that bypasses these loaders or applies additional schema inference is considered non-compliant.
- **Behavioral Rules**
 - Vectors **must** have dtype `float32`, shape `(N, D)`, and be L2-normalized within tolerance `1e-3`.
 - `vectors.meta.json` **must** contain the keys `N`, `D`, and `normalized`.
 - `metadata.parquet` **must** include column `id:int64`, sorted in ascending order and contiguous from `0..N-1` with no duplicates or gaps.

12.2 Spec-Level Validation

All harnesses and evaluation scripts **must** call the following functions from `dataio.validators`:

These functions **must** be used exactly as written. Custom validation routines, implicit coercions, or silent fallback behaviors are **not permitted**.

12.3 Run-Level Requirements

- **Oracle Sanity** – Each new artifact bucket (e.g., `v2`, `v3`) **must** be verified by running the `oracle.brute_force(...)` baseline on 5 representative queries. The exact backend output **must** match oracle results at `Recall@K = 1.0`.

Purpose	Required Function	Rule Enforced
Query vector norm	<code>ensure_unit_l2(qvec)</code>	$\ q\ _2 \approx 1$ within 1e-3
Query size	<code>validate_K(K, N)</code>	$1 \leq K \leq N$
Filter parsing	<code>parse_filters(raw)</code>	Reject malformed JSON; hard-error on failure
Filter evaluation	<code>build_allowed_ids(metadata, filters)</code>	Apply all predicates per §12 with AND semantics
Metadata schema	<code>validate_metadata_table(metadata)</code>	Required columns, dtypes, contiguity, and non-null rules

- **Immutability** – Once validated and committed, artifact buckets are **read-only**. Any regeneration requires a new version folder and updated metadata signature.
- **Compliance** – Any component that fails to use the approved loaders or validators is deemed **non-compliant** with the data interface specification.
- **Cross-artifact consistency** – The number of vectors `N` in `vectors.meta.json`, the number of rows in `metadata.parquet`, and the ID cardinality in all related artifacts **must** be identical.
- **Hybrid Index Consistency** – Any FAISS index used in a hybrid backend must be built over the validated `vectors.npy` artifact. Dimensionality (`D`) and normalization must match `vectors.meta.json`. All hybrid search iterations (e.g., progressive `nprobe`) must record their effective parameters in the logged `stats`.
- **Import Path** – All intra-project imports must use the fully qualified path from the repository root. Example:

```
from src.backends.exact_backend import ExactBackend    # correct
from .backends.exact_backend import ExactBackend      # incorrect
```

Relative imports using `.` or `..` are not permitted. This ensures consistent module resolution and independent test execution.

13. Filter Semantics

Supported operators (fixed set)

- **Equality:** `eq`, `ne`
- **Sets:** `in`
- **Ranges:** `between` (inclusive), `ge`, `le`, `gt`, `lt`
- **Text:** `like` (case-insensitive substring)
- **Geo:** `lat_between`, `lon_between`

Evaluation rules

- Predicates combined with **AND**.
- Missing values – predicate fails (row excluded).
- Type casting: numeric filters cast inputs to column dtype; cast failure = excluded. Booleans must be `0/1` or `true/false`.
- **like:** case-insensitive substring match on strings; arrays must be pipe-joined.
- **Geo:** inclusive ranges, requires both `lat` and `lon`. Coordinate system = WGS-84.
- Unknown fields/operators – **hard error**.
- Malformed JSON filter objects – **hard error** (abort run).

Selectivity

- Computed at runtime as `|allowed_id_set| / N`.
- Logged in `filter_selectivity` (see §10).

In hybrid backends, all validated filters (§12) must be enforced within the FAISS search itself via an ID selector mechanism (`faiss.IDSelectorBatch` or equivalent). Running FAISS without enforcing the validated allow-list is strictly prohibited.

Usage Rules

- All filtering and selectivity evaluation **must** use the canonical implementation in `dataio.validators.build_allowed_id_set` and `dataio.validators.validate_filters_schema(...)`. Independent re-implementations of filter logic are **prohibited**.
- Everyone must **strictly follow this contract**.
- File names and schemas are fixed—only new version folders (`v2`, `v3`) or new `run_ids` may be added.
- Any modification to the supported operator set or semantics **must** be proposed in writing and approved by the full group.